

Name: markov_chain

Inputs:

- *data*- a sequence of decimals representing previously collected data
- *order*- a positive integer representing the desired order of the markov chain

Outputs:

- a map from a decimal to the probability it appears next, that represents the Markov chain

1. **markov** $\leftarrow$ an empty map
2. **tuple_count** $\leftarrow$ an empty map
3. for each index in 0,1,2 ... , length of **data** - **order** - 1 do
 - a. **state** $\leftarrow$ a tuple with elements **data**_{index}, **data**_{index + 1}, ... , **data**_{index+order-1}
 - b. **prediction** $\leftarrow$ **data**_{index+order}
 - c. if **state** is not a key in **markov** then
 - i. **markov**(**state**) $\rightarrow$ an empty map
 - d. if **prediction** is not a key in **markov**(**state**) then
 - i. **markov**(**state**)(**prediction**) $\rightarrow$ 1
 - e. else then
 - i. **markov**(**state**)(**prediction**) $\rightarrow$ **markov**(**state**)(**prediction**) + 1
4. for each **state** in the set of keys in **tuple_count** do
 - a. **total** $\leftarrow$ 0
 - b. for each **count** in the set of values in **tuple_count**_{state} do
 - i. **total** $\leftarrow$ **total** + **count**
 - c. **probabilities** $\leftarrow$ an empty map
 - d. for each **current_state** in the set of keys in **tuple_count**_{state} do
 - i. **count** $\leftarrow$ **tuple_count**_{state}
 - ii. **probabilities**(**current_state**) $\rightarrow$ **count**/**total**
 - e. **markov**(**state**) $\rightarrow$ **probabilities**
5. Return **markov**

Name: weighted_choice

Input:

- **prob_dict**: a map between a value and its probability, where values are 0,1,2,3, probabilities are between 0 and 1, and probabilities sum to 1

Output:

- a decimal value between 0 and 3 inclusive, chosen randomly according to the probabilities
1. **rand** $\leftarrow$ a decimal between 0 inclusive and 1 exclusive chosen with a uniformly random distribution
 2. **cumulative** $\leftarrow$ 0
 3. For each **value** in the set of keys in **prob_dict** do
 - a. **cumulative** $\leftarrow$ **cumulative** += **prob_dict**(**value**)

- b. if **rand** is less than **cumulative** do
 - i. return **value**
 - c. **last_value** $\leftarrow$ **value**
 4. return **last_value**

Name: predict

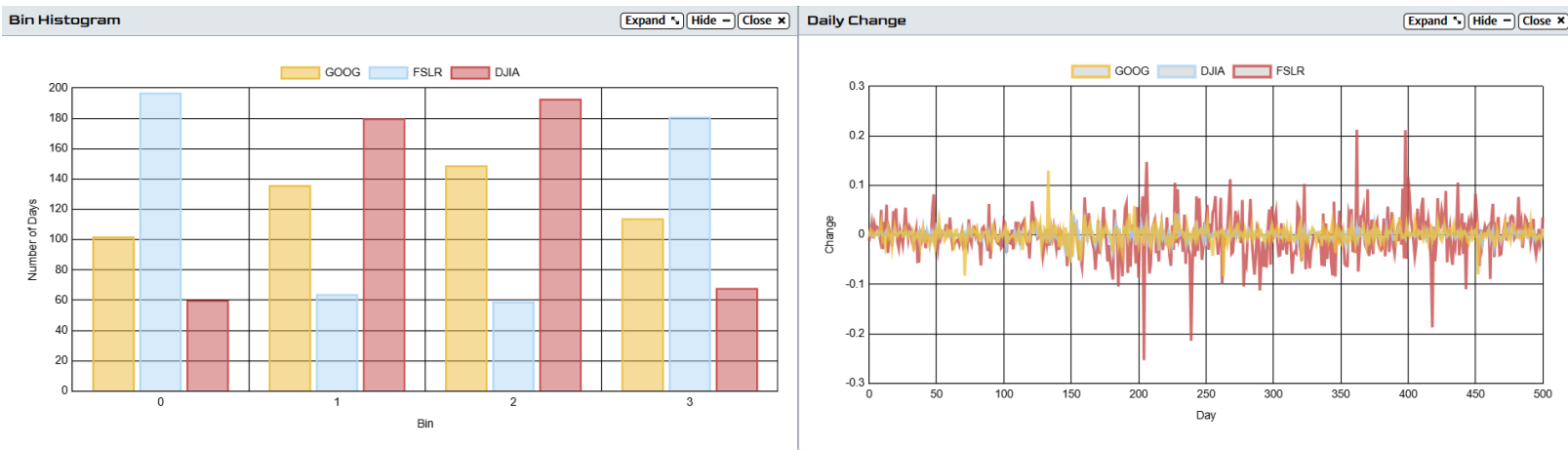
Input:

- **model**: a map representing a Markov chain
- **last**: a sequence (with length of the order of the Markov chain) representing the previous states
- **num**: an integer representing the number of desired future states

Outputs:

- a sequence of integers that are the next **num** states
1. **predictions** $\leftarrow$ an empty map
 2. for each **dummy_idx** in 0, 1, ..., **num**-1 do
 - a. if **state** is not in the set of keys of **model** then
 - i. **next_value** $\leftarrow$ a number chosen uniformly random from 0 to 3 inclusive
 - b. else then
 - i. **next_value** $\leftarrow$ **weighted_choice(model(state))**
 - c. **predictions(state)** $\rightarrow$ **next_value**
 - d. **last** $\leftarrow$ **last**₁, ..., **last**_{len(last)-1}, **next_value**
 3. returns **predictions**

Discussion



FSLR

====

Actual: [3, 0, 0, 1, 0]

Order 1 : 3.4527999999999999

Order 3 : 3.1011999999999997

Order 5 : 3.4591999999999999

Order 7 : 3.1452

Order 9 : 3.1071999999999999

GOOG

====

Actual: [1, 3, 3, 1, 1]

Order 1 : 2.2468000000000001

Order 3 : 1.508

Order 5 : 1.8139999999999999

Order 7 : 2.3035999999999997

Order 9 : 2.3156

DJIA

====

Actual: [2, 2, 2, 2, 1]

Order 1 : 0.9563999999999999

Order 3 : 0.9635999999999999

Order 5 : 0.7635999999999997

Order 7 : 1.21

Order 9 : 1.4484000000000001

- 1) For FSLR, the best order was 3. GOOG best order was 3. DJIA best order is 5. The orders might not be the same because the markov model is flawed and stock prices are affected by more factors than previous results such as macro events and federal policy. In fact, the stock market is forward looking, meaning that past data is somewhat poor of a predictor for future returns.
- 2) The DJIA can be predicted with the lowest error. This is because it has the lowest variation in day-to-day change in stock prices and its histogram is more concentrated hence making transitions more predictable than volatile stocks like FSLR.
- 3) In an n th order markov chain there are 4^n possible states since each state is a sequence of n bins, where each bin has 4 possibilities (0,1,2,3). Thus, it is 4^n .
- 4) To estimate all the possible states in an n th order Markov Chain given the training data, $4^n \leq 502$. Therefore, $n \leq 4$ since $4^5 = 1024 > 502$. However, another limitation is that even if $n \leq 4$, there is no guarantee that all of the possible states will be observed. This is because if many sequences repeat, some states may never appear and therefore the markov chain will not have a

probability for them. If there was not enough data the accuracy of the model would be worse and the average mse would be greater.